

Class Example :

Question 1 - Stack Implementation with Array:

You are required to implement a stack data structure using an **array in C**. The stack should have the following functionalities:

- a) void push(int item): This function should push an integer item onto the stack.
- b) int pop(): This function should remove and return the topmost item from the stack.
- c) int peek(): This function should return the topmost item from the stack without removing it.
- d) int isEmpty(): This function should return 1 if the stack is empty, and 0 otherwise.

Write the C code for the stack implementation with an array and demonstrate its usage by pushing a few elements onto the stack, popping elements, and checking whether the stack is empty.

Question 2 - Stack Implementation with Linked List:

You are required to implement a stack data structure using **a singly linked list in C**. The stack should have the following functionalities:

- a) void push(int item): This function should push an integer item onto the stack.
- b) int pop(): This function should remove and return the topmost item from the stack.
- c) int peek(): This function should return the topmost item from the stack without removing it.
- d) int isEmpty(): This function should return 1 if the stack is empty, and 0 otherwise.

Write the C code for the stack implementation with a linked list and demonstrate its usage by pushing a few elements onto the stack, popping elements, and checking whether the stack is empty.

Please ensure that your implementations handle edge cases appropriately and provide the correct output for different scenarios.

NB: Try to implement peek and isEmpty function at home

Question 3:

Write a C++ program that checks whether the given input string is a palindrome or not. The program should use a custom stack implementation to verify whether the string remains unchanged when read both forward and backward, considering only alphabetic characters and ignoring the character 'x' which helps to determine the mid character. If the string qualifies as a palindrome, the program should display "Palindrome" as the output. Otherwise, the program should display "Not a Palindrome" as the output.

Question 4: ([LEETCODE , 20. Valid Parentheses](#))

Given a string s containing just the characters '(', ')', '{', '}', '[', and ']', determine if the input string is valid.

An input string is valid if:

- Open brackets must be closed by the same type of brackets.
- Open brackets must be closed in the correct order.
- Every close bracket has a corresponding open bracket of the same type.

Question 5: ([Evaluation of Postfix Expression](#))

Write a C++ program that evaluates a postfix expression and returns the result. The program should take a space-separated postfix expression as input and use a stack to perform the evaluation.

For example, given the input expression "2 3 1 * + 9 -", the program should output the result "-4". The infix equivalent of this expression is "2 + (3 * 1) - 9 = 5 - 9 = -4".

Implement the necessary stack operations to handle the evaluation and conversion of the postfix expression to an infix expression, and then calculate the final result. Display the final result as the output.

Please ensure that your program handles valid input expressions and provides the correct output for different scenarios

Home Practice Example :

Question 6: ([Reverse a Stack using Recursion](#))

Write a program to reverse a stack using recursion, without using any loop.

Example:

- Input: elements present in stack from top to bottom 1 2 3 4
Output: 4 3 2 1
- Input: elements present in stack from top to bottom 1 2 3
Output: 3 2 1

Question 7: ([Implement Stack using Queues](#))

✓ Implement a last-in-first-out (LIFO) stack using only two queues. The implemented stack should support all the functions of a normal stack (push, top, pop, and empty).

Implement the MyStack class:

- void push(int x) Pushes element x to the top of the stack.
- int pop() Removes the element on the top of the stack and returns it.
- int top() Returns the element on the top of the stack.
- boolean empty() Returns true if the stack is empty, false otherwise.

Notes:

- You must use only standard operations of a queue, which means that only push to back, peek/pop from front, size and is empty operations are valid.
- Depending on your language, the queue may not be supported natively. You may simulate a queue using a list or deque (double-ended queue) as long as you use only a queue's standard operations.